

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Review Guidelines

Guidelines for peer reviewers evaluating submitted tasks.

Reviewer Checklist

As a helpful tool to use while reviewing, we have a comprehensive reviewer checklist that covers all aspects that should be looked at during a review. This includes descriptions of what we are looking for qualitatively, as well as assigns a severity level that establishes if an error for a given aspect of a submission should result in an Accept still, or if it always requires the submission to be sent back for revisions when encountered.

Go to [Reviewer Checklist](#)

Review Philosophy

As a reviewer, you're ensuring quality for the benchmark. Your goal is to:

- Catch issues before they enter the dataset
- Help contributors improve their tasks
- Maintain consistency across submissions

Remember: Each review you conduct should be comprehensive and catch ALL present errors and issues with the task. Do not simply find the first error you come across and then immediately send it back for revision. Instead, **find all issues** during your review and give feedback on all elements found.

If the EC addresses your comments, the task should be ready for acceptance, assuming no new issues appear. In other words, your feedback should be always clear, complete, and actionable.

Review Checklist

1. Read the Task Description

Authentic Prompt Styling

We have overhauled the way instructions are written. In Edition 2, the `instruction.md` file should index on **realistic prompts** that real users and engineers would use when interacting with coding agents in their daily life, and be as succinct as possible.

The instructions for every task should adhere to these six general principles:

1. Task instructions **must be concise**.
2. Task instructions **must be well specified**.
3. Task instructions **must be interesting**.
4. Task instruction **must not give answers, hints**.
5. Task instruction **must be unique**.
6. Task instruction **must use absolute paths**.

Consult the [Prompt Styling Guide](#) for further details on each core principle.

2. Review Tests

- ☐ Every requirement has a corresponding test
- ☐ Tests have informative docstrings
- ☐ Tests verify behavior, not implementation
- ☐ No brittle string matching
- ☐ No hardcoded thresholds (or thresholds are reasonable)

3. Use the Test-Quality Eval to Double-Check Your Work

There is an automated **test-quality** eval on submissions that flags common gaps so you can double-check. **It is a helper, not a replacement:** still do a full test review, and **verify every flag** — the eval will miss some issues and sometimes flag acceptable tests.

FLAG	MEANING
<code>req-gap</code>	Instruction requires something, but no test asserts it.
<code>weak-assertion</code>	A test exists but is too loose to catch wrong solutions.
<code>phantom-spec</code>	Tests enforce behavior not described in the instruction.
<code>flaky-execution</code>	A correct solution can fail due to timing, non-determinism, o
<code>vacuous-test</code>	Test can pass no matter the output (e.g. empty loops, alway

How to use it: Complete your normal test review, then go through any eval flags you might have missed.

4. Check Solution

- ☐ Solution demonstrates the process (not just outputs answer)
- ☐ Commands are deterministic
- ☐ Works in the provided environment

5. Verify Metadata

- ☐ Difficulty matches expected pass rate
- ☐ Category is appropriate
- ☐ Time estimates are realistic
- ☐ Timeout is sufficient but not excessive

6. Watch Agent Runs

Solvable vs. passing a run: Passing a run means all unit tests pass in a single run. Solvable (for the task) means that **across 10 runs**, each individual test passes at least once—so a task can have no full pass in any run and still be solvable if the agent only ever “partially” succeeds.

On the task viewer:

- Watch the terminal recording
- If agent fails, is it failing for a **good reason**?
- Check the analysis for identified issues
- Read the debug pane

Comprehensive Reviewer Checklist

- You can also use this [Reviewer Checklist](#) for a more comprehensive and detailed checklist

Common Issues to Flag

Task Description Problems

ISSUE	WHAT TO LOOK FOR
Ambiguous requirements	"Make it better", "fix the issues"
Missing output specs	Tests check files not mentioned
Relative paths	<code>./data/file.txt</code> instead of <code>/app/data/file.txt</code>
Implicit assumptions	Assumes knowledge not in instructions

Test Problems

ISSUE	WHAT TO LOOK FOR
Brittle tests	Exact string matching
Missing coverage	Requirements without tests
Order dependency	Tests that must run in sequence
Implementation testing	Parsing source code

Solution Problems

ISSUE	WHAT TO LOOK FOR
Hardcoded answers	<code>echo "42" > result.txt</code>
Non-deterministic	Uses random without seed
Incomplete	Missing steps
Over-complex	Unnecessarily convoluted

Specific Things to Watch

Testing Behavior

It is almost always better to test a behavior by actually running the code than trying to statically analyze it.

Flag: Tests that grep through source code looking for patterns.

Tool Specifications

Task instructions shouldn't mention specific tools unless there's a way to verify they were used.

Flag: "Use vim to edit the file" (can't verify vim was used).

Randomness

Tasks involving randomness MUST NOT assume the solution matches that same random order.

Flag: Tests that depend on specific `np.random.seed` values.

Test Complexity

Long test files are almost always wrong since more tests means more opportunities for error.

Flag: test_outputs.py with 20+ tests for a simple task.

Data Formats

Be especially paranoid about specifying data formats.

Flag: "Output a CSV" without specifying whether it needs headers.

Anti-Cheating

Think about how agents could cheat:

- Decompile programs to find hidden answers
- Replace programs with dummy versions
- Delete or modify tests

- Access newer git commits

Review Actions

Approve

Task is ready. No issues found.

Request Changes

Issues need fixing before acceptance. Be specific:

- What's wrong
- Where it is
- How to fix it

Good feedback:

The test `test_output_format` on line 45 uses exact string matching. Please change to check for required fields instead, allowing for formatting variations.

Bad feedback:

Tests need work.

Extra things you can include

- Point the submitter where can they found how to fix the issue or a place where they can see they are mistaken

Check the New: Rubrics section in the trainig site

Decline

Fundamental issues that can't be easily fixed:

- Too easy (worst-model accuracy consistently above 80%)
- Essentially duplicate of existing task
- Core concept is flawed

Explain clearly why, and whether revision could salvage it.

Next Steps

- [Reviewer Checklist](#) — Full high/medium/low acceptance criteria for reviews
- [Reviewer Training](#) — Training videos and materials for reviewers
- [Rubrics](#) — How to write and review rubrics for evaluating agent traces
- [Quality Guidelines](#) — Required quality standards for all tasks
- [Common errors to watch for](#)

PREVIOUS

[Agent Review Reference](#)

NEXT

[Reviewer Checklist](#)